

UNIVERSITY OF COLORADO BOULDER

ASEN 5264: DECISION MAKING UNDER UNCERTAINTY

---

# **Residual Reinforcement Learning for Terminal Missile Guidance**

---

**Jonathan Morris**



Ann and H.J. Smead  
Aerospace Engineering Sciences

UNIVERSITY OF COLORADO **BOULDER**

## CONTENTS

|            |                                                                                    |   |
|------------|------------------------------------------------------------------------------------|---|
| <b>I</b>   | <b>Introduction</b>                                                                | 2 |
| <b>II</b>  | <b>Background and Related Work</b>                                                 | 2 |
| <b>III</b> | <b>Problem Formulation</b>                                                         | 2 |
| III-A      | State Action and Reward . . . . .                                                  | 2 |
| III-B      | Equations of Motion / Transition . . . .                                           | 3 |
| III-C      | Target Motion . . . . .                                                            | 3 |
| <b>IV</b>  | <b>Solution Approach</b>                                                           | 3 |
| IV-A       | Training Algorithm . . . . .                                                       | 3 |
| IV-B       | Exploration . . . . .                                                              | 3 |
| IV-C       | Model Architecture and Parameters . .                                              | 4 |
| IV-D       | Training Environment . . . . .                                                     | 4 |
| IV-E       | Curriculum . . . . .                                                               | 4 |
| IV-F       | Summary of Model Parameters . . . . .                                              | 5 |
| <b>V</b>   | <b>Results and Discussion</b>                                                      | 5 |
| V-A        | Evaluation Environment 1: Less Missile<br>Energy, Increased Target Maneuverability | 5 |
| V-B        | Evaluation Environment 2: Simple<br>Straight-Flying Target . . . . .               | 5 |
| V-C        | Evaluation Environment 3: Highly Ma-<br>neuverable Targets . . . . .               | 6 |
| V-D        | Evaluation: Single Flight Example . . .                                            | 6 |
| <b>VI</b>  | <b>Future Work and Conclusion</b>                                                  | 6 |
| <b>VII</b> | <b>Contributions and Release, Algorithm Usage</b>                                  | 7 |
|            | <b>References</b>                                                                  | 7 |

## NOMENCLATURE

| Symbol             | Description                                                                                     |
|--------------------|-------------------------------------------------------------------------------------------------|
| $a_{targ}$         | target acceleration vector (m/s <sup>2</sup> )                                                  |
| $\mathbf{a}_{apn}$ | acceleration commanded by APN                                                                   |
| $\mathbf{a}_{NN}$  | acceleration commanded by Agent                                                                 |
| $C_D$              | drag coefficient (dimensionless)                                                                |
| $C_{D,0}$          | zero lift drag coefficient                                                                      |
| $C_N$              | normal force coefficient (dimensionless)                                                        |
| $k_1$              | induced drag factor                                                                             |
| $m$                | mass of the vehicle (kg)                                                                        |
| $r$                | range                                                                                           |
| $v_{rel}$          | relative velocity                                                                               |
| $h$                | current altitude above surface of earth                                                         |
| $g_{lim}$          | scalar factor between (0-1) used in model training to de-<br>of allowable acceleration commands |
| $r_{CPA}$          | Range at closest point of approach (m)                                                          |
| $N_{APN}$          | APN scalar gain                                                                                 |
| $q$                | dynamic pressure                                                                                |
| $ZEM$              | zero effort miss                                                                                |
| $\mathbf{y}$       | denotes relative range to target                                                                |
| $\mathbf{r}^E$     | position vector in ECEF frame                                                                   |
| $\lambda$          | Line-of-sight angle (rad)                                                                       |
| $\mathbf{J}^B$     | denotes vector J in body frame                                                                  |
| $\mathbf{J}^E$     | denotes vector J in ECEF frame                                                                  |

## I. INTRODUCTION

Residual reinforcement learning is a technique that involves splitting a problem into a portion that can be solved by traditional methods, and another ‘residual’ part that is solved with reinforcement learning. This paper examines applying this concept to a simple missile guidance problem. The approach uses augmented proportional navigation (APN) as a baseline command and an agent trained with proximal policy optimization (PPO) augments this command. Improved performance in terms of impact distance is shown over the baseline of pure APN.

## II. BACKGROUND AND RELATED WORK

Terminal missile guidance is an area in which residual reinforcement learning can be/has been applied. Typical approaches involve simple classical proportional navigation or model based optimal control/guidance approaches. Classical approaches are limited in their capability, especially against highly maneuverable targets. Model based approaches are highly sensitive to model parameters. Residual reinforcement learning can be applied to this problem, relying on simple classical control as a baseline, and eliminating model errors by training in a stochastic environment. In this paper APN is used with  $N_{APN} = 3$  because it can be shown to be mathematically optimal for minimizing control effort with certain assumptions (e.g., absence of drag, stationary target, etc.) [6]. The goal of the residual agent is then to improve the results from this baseline command.

This exact concept is explored in this paper where the classical control law, Augmented Proportional Navigation (APN) is used and RL attempts to minimize the residual. A highly relevant example of this is outlined in the following paper: *A Learning-Based Computational Impact Time Guidance* (Liu et al., 2021)[4]. In this paper the authors utilize residual reinforcement learning in order to control impact time. Their general setup is to use a DNN to estimate  $t_{go}$  then feed use estimate to a PPO trained actor-critic agent that adds a bias control vector to a proportional navigation control vector in order to control the actual impact time.

Residual RL has been applied to many different problems areas, and is especially well suited to problems that have strong baseline solutions that require complex modeling to solve completely. An example of this is a robotic arm, developing control laws to actuate the arm in different manners is a simple problem, but correctly applying these movements to complex tasks involves great detail in modeling (i.e, grip strength, friction on the robotic joints, contact with external objects). An oft cited and seminal paper on this exact topic is *Residual Reinforcement Learning for Robot Control* (Johannink et al., 2019)[3] wherein a standard proportional control was applied to a complex dynamical system involving a robot arm. A standard closed loop control was augmented with a residual policy learned by deep reinforcement learning (trained with TD3, similar to DDPG), and the combined control was able to manipulate physical objects without manual tuning of the control law.

Another similar example is *ProxFly: Robust Control for Close Proximity Quadcopter Flight* (Zhang et al., 2024) [7] where a traditional flight controller was augmented with an RL trained agent that compensates for complex flow conditions caused by quadcopters flying in close proximity. Their implementation was very similar to the implementation in this paper as they used a classic control and augmented it with a PPO trained agent.

Finally, many other non-classical approaches involve exclusively agent control (i.e., no baseline command). *Reinforcement learning-based missile terminal guidance of maneuvering targets with decoys* (Deng et. al.) is a paper that does this using a version of DDPG[1].

## III. PROBLEM FORMULATION

The problem is formulated as a partially observable Markov decision process. The agent receives perfect measurements in its state, but the process is partially observable from the agents perspective as it does not know how the target will maneuver.

### A. State Action and Reward

All terms in the agent’s state are all in body coordinates. The state includes a line of sight vector from it to the target, the time derivative of the line of sight vector, the range to target, relative velocity, target’s acceleration, altitude, and its air relative velocity. It also has access to a scalar value that maps the magnitude of its current allowed magnitude of acceleration command to the its max value. This was only used for curriculum training and is set to 1.0 (its max value) when evaluated. This will be discussed in more detail in later sections.

The action space is simply an acceleration vector in the directions normal to the long axis of the vehicle. This action is then added to the baseline APN command. Note for this project the agent is limited to  $\pm 2$  g of total control. This is because it is only mean to augment the APN control.

The reward structure is intentionally sparse, with only a very small control penalty being applied at every step, and otherwise terminal rewards. The terminal reward is scaled such that it rewards mostly for accuracy, and then additionally for impact velocity if the flight was sufficiently accurate. The sparse award is made possible by the residual structure where the baseline APN is frequently able to achieve ‘hits’ without an input from the agent. Although the only goal of the system is to achieve higher hit rates, the velocity was an important part in shaping the reward. This was because as the agent learned to shape its trajectories in ways that increased impact velocity (and thus conserve energy throughout the flight), it had more available energy to pursue targets that would otherwise be impossible to hit. The discounting parameter,  $\gamma$ , was set to 1.0 as the reward was almost exclusively terminal. Note that the parameter  $v_{expected}$  was used to normalize the impact velocity vs an APN baseline. This was computed by fitting a surface to data from a sweep of the initial altitude, range, and initial missile velocity. This was done in an attempt to ensure comparable ‘success’ had comparable rewards, regardless of

initial condition. This  $v_{expected}$  does not account for initial target amplitude.

The state, action, and reward are defined below:

$$\begin{aligned} S &= \{\lambda^B, \dot{\lambda}^B, r, v_{rel}, \mathbf{a}_{targ}^B, h, a_{lim}\} \\ A &= [a_{nn,y}^B, a_{nn,z}^B] \\ R_t &= \begin{cases} B & \text{if Terminal } (r_{CPA} < 1000) \\ C & \text{if Terminal (Miss/Crash)} \\ E & \text{otherwise} \end{cases} \end{aligned}$$

Where,

$$B = 100 \cdot e^{-\frac{r_{CPA}}{10.0}} + 100 \cdot \left( \frac{\|\mathbf{v}_m\|}{v_{expected}} \right) e^{-\frac{r_{CPA}}{2.0}}$$

$$C = -100$$

$$E = -1e - 4(\|A\|)^2$$

**Terminal States:** Crash:

$$\mathbf{r}^E < R_{EARTH}$$

Increasing range to target:

$$\frac{dr}{dt} > 0$$

For clarity, note that the increasing range to target is triggered for both a successful intercept and a miss (e.g., missile passes within 0.1m of target- $\hat{z}$  continues flying past- $\hat{z}$  terminal state).

### B. Equations of Motion / Transition

The transition of the MDP is modeled as follows, where the forces acting on the vehicle are summed, converted to acceleration, and integrated using a fourth order Runge Kutta method. Its important to note that in this simple problem formulation the vehicle has no thrust and is simply gliding. Additionally, note that the drag increases with the square of the force produced by the fins to approximate induced drag. Finally, the problem was modeled in an earth centered earth fixed inertial frame, and the vehicle's body frame. Thus the fictitious centrifugal and coriolis forces are modeled (in the scale of the problem though, they are practically negligible). The forces acting on the vehicle are as follows:

$$\begin{aligned} \sum F &= -mg \frac{\mathbf{r}^E}{\|\mathbf{r}^E\|} - C_D q A_{ref} \frac{\mathbf{x}^B}{\|\mathbf{x}^B\|} + (\mathbf{a}_{apn}^B + \mathbf{a}_{NN}^B) \\ C_D &= C_{D,0} + k_1 C_N^2 \\ C_N &= \min(10.0, \left( \frac{m\|(\mathbf{a}_{apn}^B + \mathbf{a}_{NN}^B)\|^2}{q A_{ref}} \right)) \\ q &= \frac{1}{2} \rho v_m^2 \end{aligned}$$

The augmented proportional navigation baseline commanded acceleration is computed based on the zero effort miss form from *Tactical and Strategic Missile Guidance* [6]. Note

in the following  $t_{go}$  is the estimated time to impact and is calculated as  $t_{go} = r/v_c$ .

$$ZEM = \mathbf{y}^E + \dot{\mathbf{y}}^E t_{go} + \frac{1}{2}(\mathbf{a}_{targ}^E + \mathbf{g}^E) t_{go}^2$$

$$\mathbf{a}_{apn} = -N_{APN} \frac{ZEM}{t_{go}^2}$$

The implementation also limits the total commanded acceleration (to model structural limits), and command remains constant when  $t_{go}$  is less than 0.25s (to avoid infinite command input).

### C. Target Motion

The approach was evaluated against maneuvering targets. The maneuvering targets were able to initiate a sinusoidal lateral acceleration command at a random distance, random amplitude, with a random period, and number of cycles. The agent has none of this information and thus the problem is partially observable to the agent.

## IV. SOLUTION APPROACH

### A. Training Algorithm

To solve this problem an agent was trained using proximal policy optimization. Proximal policy optimization is an on-policy algorithm that limits weight updates. This is a very common RL algorithm and was chosen for this project for many reasons. One reason is that PPO allows for continuous action spaces, crucial for the guidance problem. Additionally, being on policy allows for curriculum training where the agent can slowly be given more control authority or more complex environments. Finally, clipping of the surrogate objective is extremely useful as well as it limits model updates that could collapse the policy.

### B. Exploration

The exploration policy was one of the most crucial aspects of the project. When exclusively using the standard entropy regularized exploration, the agent failed to learn due to the high frequency noise (episodes were 0.1s) of exploration. It was thus decided to use the implementation of state dependent exploration to more smoothly output actions. Rather than sample a random noise vector from a gaussian distribution for exploration noise, state dependent exploration defines a random noise matrix ( $\theta_\epsilon$ ) at the start of every episode (or fixed number of steps). This is then mapped to a state through the 'latent features' of the neural network (last hidden layer,  $z_\mu$ ), such that the noise at each time step is now a function of state. This creates smooth exploration because the state evolves smoothly, and thus the latent features do as well. Mathematically, this is  $\mathbf{a} = \pi(\mathbf{s}|\theta) + \theta_\epsilon \mathbf{z}_\mu(\mathbf{s})$ . The actual implementation of this state dependent exploration is the version of gSDE implemented in stable baselines 3 (SB3), based on the paper *Smooth Exploration for Robotic Reinforcement Learning* (Raffin et al.) [5].

### C. Model Architecture and Parameters

The model architecture used for this problem is an actor-critic based approach where the actor's policy gradient (Eq 1) is updated based on the temporal difference residual ( $r_k + \gamma V_\phi(s_{k+1}) - V_\phi(s_k)$ ) supplied from the critic's Value estimation ( $V_\phi(s_k)$ ) calculated from minimizing the loss function (Eq 2).

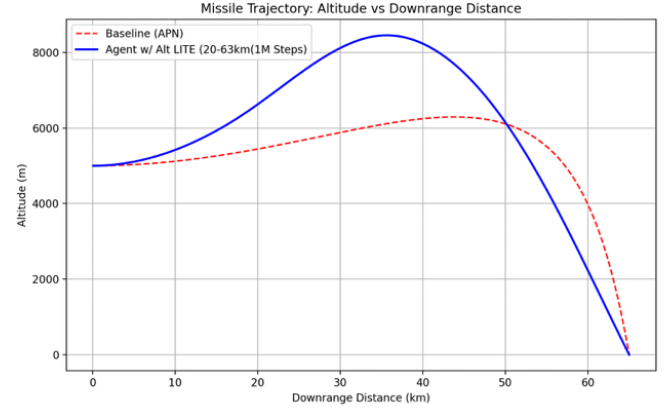
$$\nabla U(\theta) = E_\tau \left[ \sum_{k=0}^d \nabla_\theta \log \pi_\theta(a_k | s_k) \gamma^k (r_k + \gamma V_\phi(s_{k+1}) - V_\phi(s_k)) \right] \quad (1)$$

$$l(\phi) = E \left[ (V_\phi(s) - \hat{V}^{\pi_\theta}(s))^2 \right] \quad (2)$$

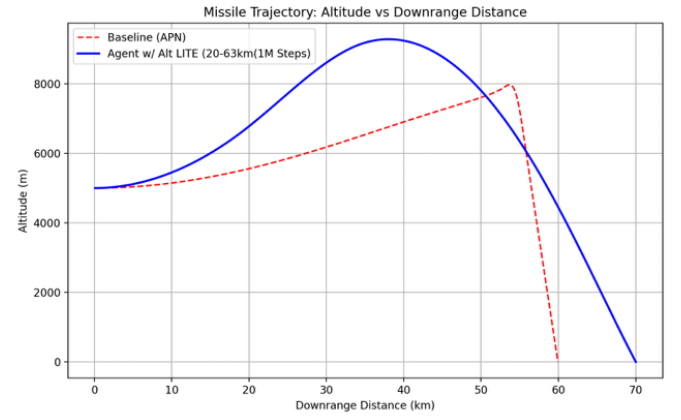
Recall that the agent has no information about the targets planned maneuvers besides what it can observe in its state/observation vector. To alleviate this, frame stacking was used to give the agent the current observation and past observations as well. A small sweep of number of frames stacked and number of nodes in 2 hidden layers for the actor and critic was used to inform a reasonable number of frames and nodes. Stacking 4 frames with both actor and critic having 128 nodes in 2 hidden layers provided acceptable results and training speeds.

### D. Training Environment

From previous study against stationary targets, it was found that an efficient way to train the agent was to train against an environment where the APN was already successful, and then by shaping the reward function you could demonstrate success against more 'difficult' targets. An example of this is 1, where the agent was trained within the range where APN was successful, but after training was able to successfully hit targets well outside the range of just the baseline APN. Pictured are the flight profiles at 65km and 70km target range. Note that APN essentially stalls the vehicle, whereas the APN + Agent has learned to 'loft' its trajectory in a way that preserves kinetic energy throughout the flight.



(a) Side View of 65km Flight Profiles



(b) Side View of 70km Flight Profiles

Fig. 1: Stationary Target Example

This approach was then applied to the more complex environment with maneuvering targets. A sweep of randomized target parameters found an acceptable range where the baseline APN recorded hits ( $\geq 1.0$ m closest distance) on greater than 95% of flights. This initial training environment is described in Table II.

TABLE II: Simulation Environment Initialization Bounds

| Parameter                        | Description               | Uniform Bounds     | Units |
|----------------------------------|---------------------------|--------------------|-------|
| <i>Velocities &amp; Geometry</i> |                           |                    |       |
| $v_m$                            | Missile Initial Velocity  | [700.0, 900.0]     | m/s   |
| $v_t$                            | Target Initial Velocity   | [40.0, 80.0]       | m/s   |
| $r_{init}$                       | Initial Range to Target   | [10000.0, 15000.0] | m     |
| $\Delta h$                       | Target Altitude Offset    | [0.0, 1000.0]      | m     |
| $\theta_{targ}$                  | Target Initial Azimuth    | [160.0, 200.0]     | deg   |
| <i>Target Evasive Maneuvers</i>  |                           |                    |       |
| $r_{manu}$                       | Maneuver Initiation Range | [6000.0, 9000.0]   | m     |
| $a_{targ}$                       | Target Maneuver Amplitude | [1.0, 3.0]         | $g$   |
| $T_{manu}$                       | Target Maneuver Period    | [2.0, 10.0]        | s     |
| $N_{cycles}$                     | Target Maneuver Cycles    | [1.0, 10.0]        | N/A   |

### E. Curriculum

It was found that against moving targets, it was difficult to have the agent learn immediately with its maximum control

authority ( $\pm 2g$ ) as such, a curriculum was implemented where the agent started with  $\sim 1g$  of authority and this was linearly increased to  $2g$  at a percent of the maximum number of training steps. This was accomplished by clipping the output of the neural net based on the progress of training ( $a_{nn} = \text{clip}(a_{nn,raw}, -a_{lim,curr}, a_{lim,curr})$ ). This was necessary as it ensured an output mapped from the neural net always had the same physical meaning (i.e,  $a_{nn,y} = 0.1$  is always 1 tenth of the maximum allowed output)% . Otherwise, without this scaling, even a very slow increase in the allowable  $a_{lim}$  caused the training to collapse.

### F. Summary of Model Parameters

Many different model parameters were examined, but the most successful trained model had these parameters. The cyan line in the figures shown was the most successful trained model (Figures 2 3). Note that all three of these models perform comparably, in terms of minimizing hit distance, but the cyan was generally slightly better. The evaluation was a deterministic simulation with 100 episodes every 250,000 steps in the environment. Note that this was conducted with  $a_{lim} = 1.0$  even if the agent was not yet to this point.

TABLE III: Summary of Parameters

| Parameter                | Value                  | Notes                   |
|--------------------------|------------------------|-------------------------|
| Learning Rate            | 3e-4                   | constant                |
| Actor Structure          | 128,128                | N/A                     |
| Critic Structure         | 128,128                | N/A                     |
| State                    | 52x1                   | (13,1) state, 4 stacked |
| Entropy Coef             | 0.005                  | small b/c gSDE          |
| gSDE sample freq         | 20 steps               | 2s in environment       |
| Batch Size               | 1024                   | N/A                     |
| Number of Steps          | 16386                  | $\sim 10$ episodes      |
| Initial $a_{lim}$        | 10 m/s <sup>2</sup>    | N/A                     |
| Max $a_{lim}$            | 19.62 m/s <sup>2</sup> | 2 g                     |
| Max $a_{lim}$ step count | 20,000,000             | 67% of total steps      |
| Total Steps              | 32,000,000             | N/A                     |
| Total Episodes           | 183,000                | Approximate             |

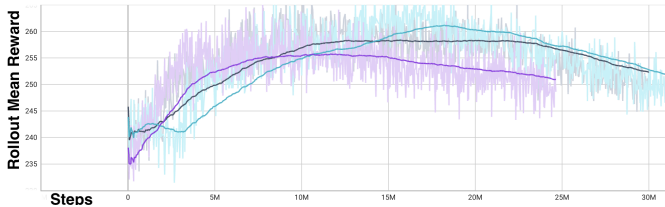


Fig. 2: Rollout Reward Training Curves

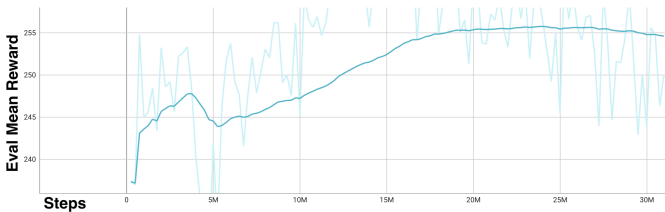


Fig. 3: Evaluation Reward Training Curves

## V. RESULTS AND DISCUSSION

The results were compared against multiple evaluation environments, and more strenuous bounds were applied to the evaluation environment as compared to the training environment. Monte Carlo simulations were conducted for 1000 samples in each of these environments, and the results were compared.

### A. Evaluation Environment 1: Less Missile Energy, Increased Target Maneuverability

The first evaluation environment aimed to reduce the missiles initial energy while simultaneously increasing the target's maximum maneuverability. The initial parameters and results of this are documented in Tables IV and V. Note that the agent+APN performs much better than the APN baseline, **achieving impact within the 1m target 14% more often.**

TABLE IV: Evaluation Environment 1 Initialization Bounds

| Parameter                        | Description               | Uniform Bounds    | Units |
|----------------------------------|---------------------------|-------------------|-------|
| <i>Velocities &amp; Geometry</i> |                           |                   |       |
| $v_m$                            | Missile Initial Velocity  | [600.0, 900.0]    | m/s   |
| $v_t$                            | Target Initial Velocity   | [40.0, 80.0]      | m/s   |
| $r_{init}$                       | Initial Range to Target   | [8000.0, 15000.0] | m     |
| $\Delta h$                       | Target Altitude Offset    | [0.0, 1000.0]     | m     |
| $\theta_{targ}$                  | Target Initial Azimuth    | [160.0, 200.0]    | deg   |
| <i>Target Evasive Maneuvers</i>  |                           |                   |       |
| $r_{manu}$                       | Maneuver Initiation Range | [6000.0, 9000.0]  | m     |
| $a_{targ}$                       | Target Maneuver Amplitude | [1.0, 5.0]        | $g$   |
| $T_{manu}$                       | Target Maneuver Period    | [2.0, 10.0]       | s     |
| $N_{cycles}$                     | Target Maneuver Cycles    | [1.0, 10.0]       | —     |

TABLE V: Performance Metrics, Evaluation Environment 1

| Metric                 | Baseline APN | APN + Agent |
|------------------------|--------------|-------------|
| Hit Rate ( $< 0.5$ m)  | 50.10%       | 66.80%      |
| Hit Rate ( $< 1.0$ m)  | 72.90%       | 86.20%      |
| Hit Rate ( $< 10.0$ m) | 88.4%        | 97.1%       |
| Mean Miss Distance     | 19.65 m      | 1.70 m      |
| Max Miss Distance      | 1069.53 m    | 138.61 m    |

### B. Evaluation Environment 2: Simple Straight-Flying Target

Another test was done with the same initial bounds as environment 1, but without the target maneuvering. This was to see if the neural network had perhaps been overfit to the target's maneuvering and perform worse against targets moving in a straight line. This was not the case, and the results are shown in Table VI. The Agent +APN performs identically to the APN alone (within the bounds of error on the 1000 sample Monte Carlo Simulation).

TABLE VI: Performance Metrics, Evaluation Environment 2

| Metric                | Baseline APN | APN + Agent |
|-----------------------|--------------|-------------|
| Hit Rate ( $< 0.5$ m) | 100%         | 100%        |
| Mean Miss Distance    | 0.16 m       | 0.16 m      |
| Max Miss Distance     | 0.39 m       | 0.37 m      |

### C. Evaluation Environment 3: Highly Maneuverable Targets

One final evaluation was conducted to show performance exclusively against highly maneuverable targets. These targets initiate their maneuvers at the same distance, but are much more maneuverable than the targets in the training environment. The results are shown in Tables VII and VIII. **Note that the APN + Agent outperforms the baseline APN by 16%** at the target hit rate of 1.0m, but what is more interesting is the hit rate inside of 10m being nearly 30% greater than that of the baseline APN. This was evaluated further in Figure 5. Note the axes are scaled the same for both plots. You can tell from both plots that more maneuverable targets were generally more difficult to hit, and targets that started further away were harder to hit as well, but the APN+Agent vastly outperformed the baseline APN here. The fact that the hit rate inside of 10m was 78% whereas the hit rate inside of 1.0m was only 36% implies that some additional training in an environment like this could improve the model further.

TABLE VII: Evaluation Environment 3

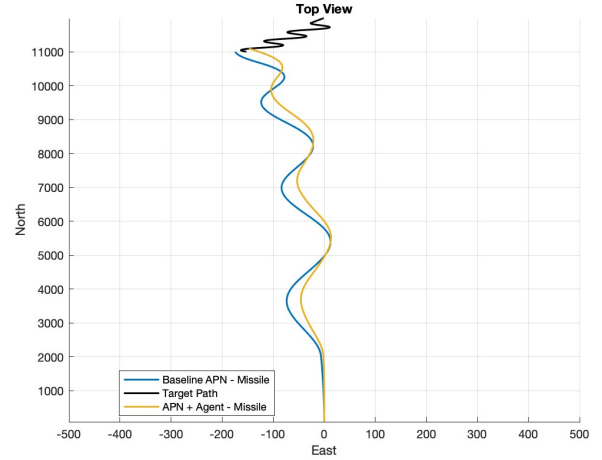
| Parameter                        | Description               | Uniform Bounds    | Units |
|----------------------------------|---------------------------|-------------------|-------|
| <i>Velocities &amp; Geometry</i> |                           |                   |       |
| $v_m$                            | Missile Initial Velocity  | [700.0, 900.0]    | m/s   |
| $v_t$                            | Target Initial Velocity   | [40.0, 80.0]      | m/s   |
| $r_{init}$                       | Initial Range to Target   | [8000.0, 12000.0] | m     |
| $\Delta h$                       | Target Altitude Offset    | [0.0, 1000.0]     | m     |
| $\theta_{targ}$                  | Target Initial Azimuth    | [160.0, 200.0]    | deg   |
| <i>Target Evasive Maneuvers</i>  |                           |                   |       |
| $r_{manu}$                       | Maneuver Initiation Range | [6000.0, 9000.0]  | m     |
| $a_{targ}$                       | Target Maneuver Amplitude | [5.0, 8.0]        | $g$   |
| $T_{manu}$                       | Target Maneuver Period    | [2.0, 10.0]       | s     |
| $N_{cycles}$                     | Target Maneuver Cycles    | [1.0, 10.0]       | —     |

TABLE VIII: Performance, Evaluation Environment 3

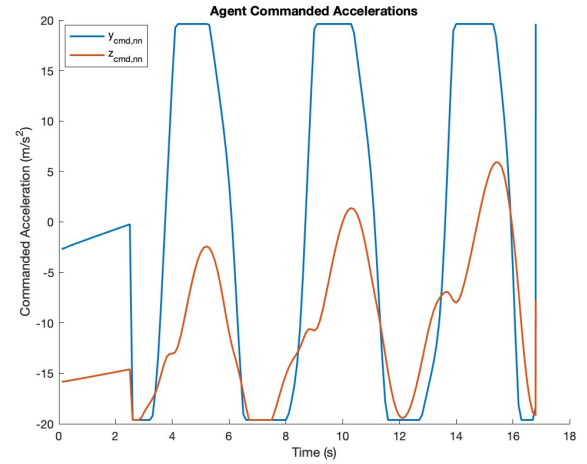
| Metric              | Baseline APN | APN + Agent |
|---------------------|--------------|-------------|
| Hit Rate (< 0.5 m)  | 17.1%        | 24.1%       |
| Hit Rate (< 1.0 m)  | 20.9%        | 36.3%       |
| Hit Rate (< 10.0 m) | 48.3%        | 77.8%       |
| Mean Miss Distance  | 98.6 m       | 14.8 m      |
| Max Miss Distance   | 1824.9 m     | 534.8 m     |

### D. Evaluation: Single Flight Example

Figure 4 demonstrates what the agent has learned. In this example, against a target maneuvering at a constant 5 g, the APN missed by 22.0 m whereas the agent+APN missed by 1.01 m. You can see that the agent basically learned to damp out the side to side ‘chasing’ of the target by the APN. Note that the axes in the North-East plot are not equal to make this damping plainly visible.



(a) ‘Damping’ of the APN commands, Top Down View

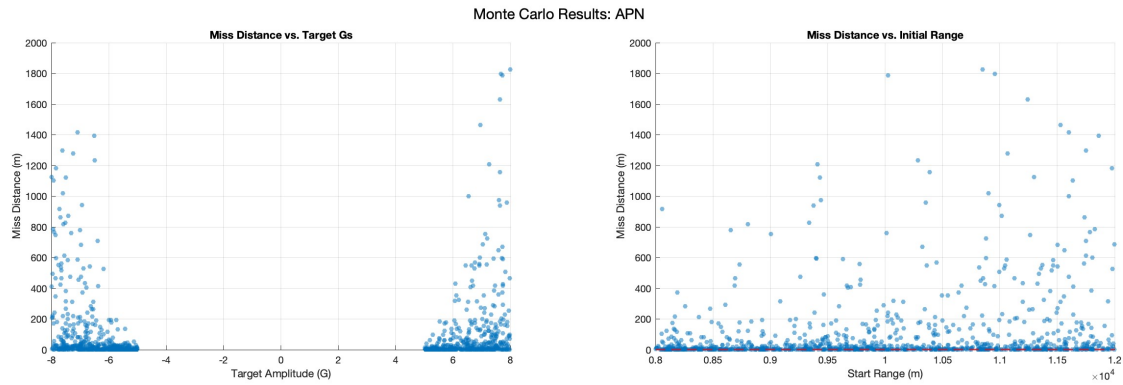


(b) Commanded accelerations by the Agent

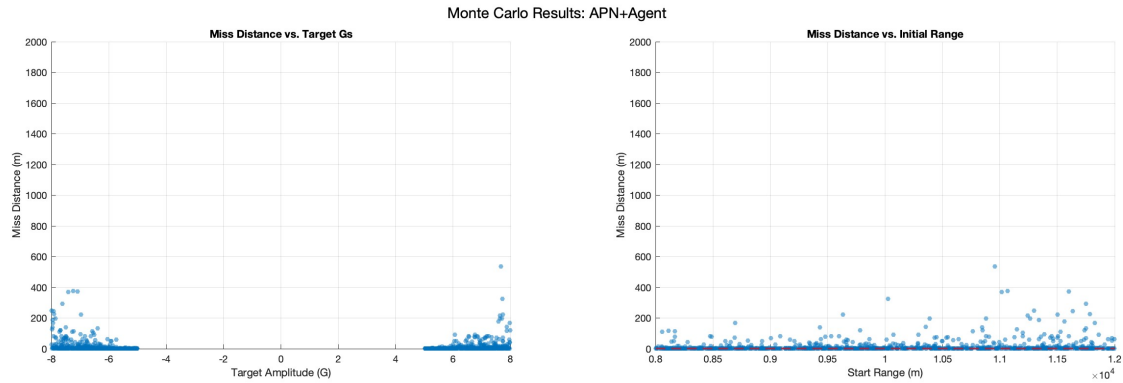
Fig. 4: Damping of APN Commands

## VI. FUTURE WORK AND CONCLUSION

Overall this paper shows that residual RL is a technique applicable to problems like terminal missile guidance. In general the solution performed better than the baseline, although I believe based on some of the results already discussed (high hit rate inside 10m, low hit rate inside 1m) implies that even with the current approach, more training in increasingly complex environments could be beneficial. If I were to continue this work I would first try and evaluate the current best model against maneuvers it hasn’t seen, not simply more strenuous sinusoidal acceleration commands. It seems the agent hasn’t been overfit to these maneuvers as it still performs as good as the Baseline APN for straight-flying targets, but more investigation is warranted. Additionally, it would be interesting to see how sensitive the agent is to modeling errors. For example, the  $C_D$ ,  $k_1$  (induced drag factor), mass and other vehicle constants were all kept constant in training. It would be interesting and important to see how sensitive the model is to each of these parameters. This paper has shown that this approach is



(a) APN Monte Carlo Results



(b) APG + Agent Monte Carlo Results

Fig. 5: Impact of Target Initial Amplitude and Initial Range to Target on Miss Distance

successful, and continuing to pursue this with more complex problems such as the line of sight curvature approach in the paper *Line of Sight Curvature for Missile Guidance using Reinforcement Meta-Learning* (Gaudet and Fufaro, 2022) [2] where the authors used a recurrent neural network to bias the line of sight vector that the vehicle observes. Again, overall this paper shows the benefits of residual RL with actor-critic based PPO for a problem of this type.

## VII. CONTRIBUTIONS AND RELEASE, ALGORITHM USAGE

I completed the entire project myself. I (the author) grant permission for this report to be posted publicly. Code is available at my github ([https://github.com/jonathanmorris899/RL\\_ResidualMissileGuidance\\_DMU.git](https://github.com/jonathanmorris899/RL_ResidualMissileGuidance_DMU.git)).

I used stable-baselines-3 (sbx) and gymnasium for PPO and my simulation environment. I used numpy, and pandas for generic python and saving csvs. The c++ simulation code was all custom and it was originally intended for doing RL for exoatmospheric intercepts. The only c++ libraries I used was Eigen for a lot of the vector math and pybind to link it to python (and built-in c++/python libraries as well).

## REFERENCES

- [1] Tianbo Deng, Hao Huang, Yangwang Fang, Jie Yan, and Haoyu Cheng. Reinforcement learning-based missile terminal guidance of maneuvering targets with decoys. 2023.
- [2] Brian Gaudet and Robert Fufaro. Line of sight curvature for missile guidance using reinforcement meta-learning, 2022.
- [3] Tobias Johannink, Shikhar Bahl, Ashvin Nair, Jianlan Luo, Avinash Kumar, Matthias Loskyll, Juan Aparicio Ojea, Eugen Solowjow, and Sergey Levine. Residual reinforcement learning for robot control, 2018.
- [4] Zichao Liu, Jiang Wang, Shaoming He, Hyo-Sang Shin, and Antonios Tsourdos. A learning-based computational impact time guidance. *arXiv*, 2021.
- [5] Antonin Raffin, Jens Kober, and Freek Stulp. Smooth exploration for robotic reinforcement learning, 2021.
- [6] Paul Zarchan. *Tactical and Strategic Missile Guidance*. Progress in Astronautics and Aeronautics. American Institute of Aeronautics and Astronautics, Reston, VA, 6th edition, 2012.
- [7] Ruiqi Zhang, Dingqi Zhang, and Robert Mueller. Proxify: Robust control for close proximity quadcopter flight via residual reinforcement learning, 2024.